



**SCHOOL OF EXECUTIVE EDUCATION
AND LIFELONG LEARNING**

Bank Account Fraud Analytics

An AI and ML Capstone Project: Finance Detect fraudulent transactions

Submitted by: Richard Ferdinand A. De Roxas

Program: Postgraduate Diploma in AI and Machine Learning

Adviser: Professor Emeritus Carmen Tagliente

Date of Submission: July 2026

1. Problem Statement

The Business Problem

Financial institutions face massive losses from bank account opening fraud (e.g., synthetic identities, identity theft, or mule accounts). However, over-correcting by setting hyper-sensitive fraud filters creates **user friction**—legitimate customers experience manual verification delays, transaction declines, and application drop-offs, directly damaging customer acquisition metrics and brand trust.

The primary business goal is to deploy an automated risk-assessment system that instantly screens new accounts at onboarding. The system must maximize fraud detection while strictly capping the negative impact (friction) on legitimate applicants and keeping operational review costs manageable.

The Data Science Problem

Given a large-scale dataset of historical bank account applications spanning 8 months, build a machine learning model that predicts the likelihood of an incoming application being fraudulent ($\text{fraud_bool} = 1$). To be useful in production, this model must address three distinct technical hurdles:

1. **Extreme Class Imbalance:** Fraudulent applications represent only about **1.10%** of the entire dataset, meaning standard classifiers will naturally default to predicting "legitimate" to maximize basic accuracy.
2. **Temporal Covariate Shift:** Fraud patterns and legitimate behaviors naturally morph over time, requiring robust generalization across the 8-month timeline.
3. **Demographic Bias:** The model must not unfairly penalize specific protected demographic groups (such as younger applicants, using the sensitive attribute age).

2. Task Type

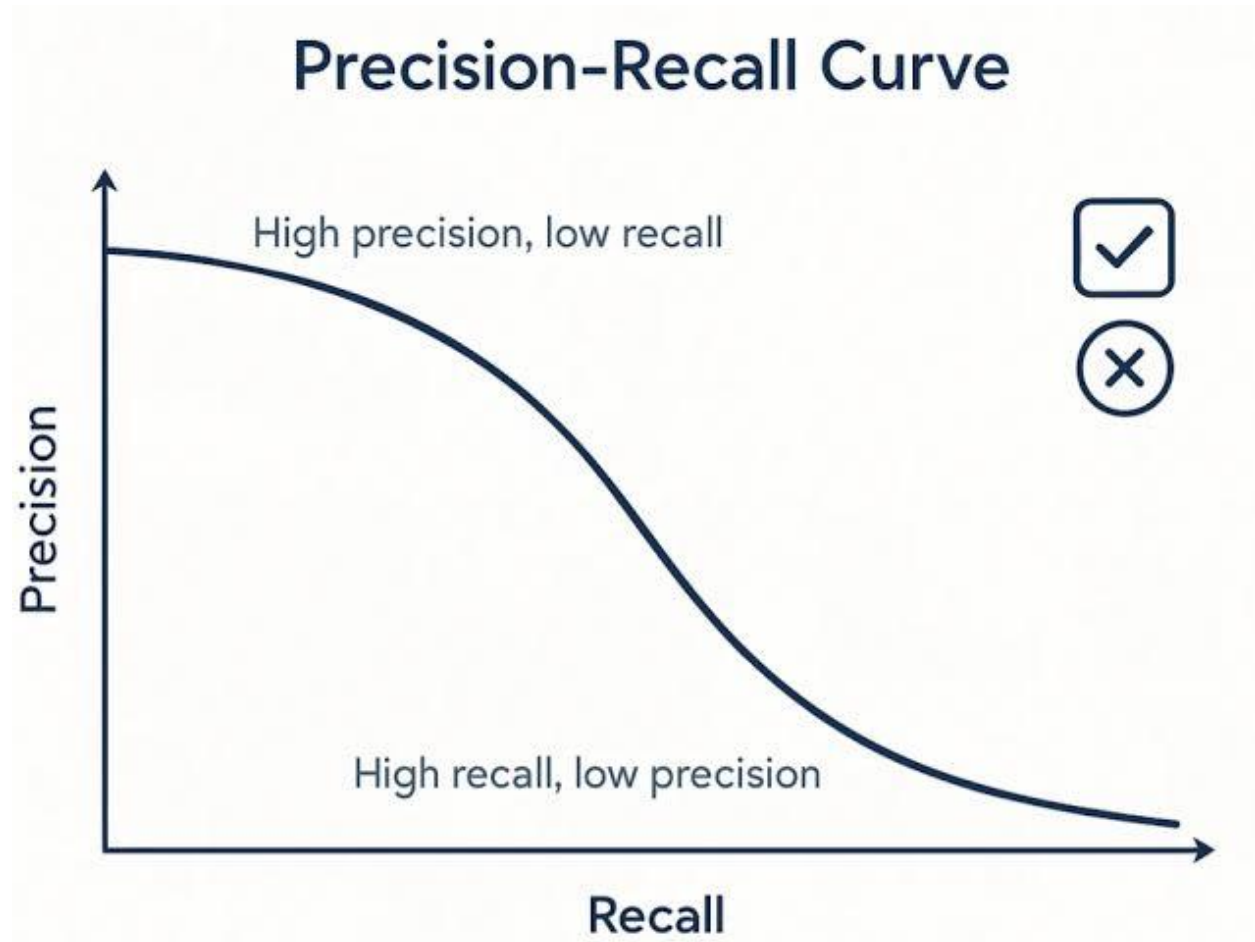
This is a **Supervised Binary Classification** task.

- **Input Features (X):** Tabular application data (applicant income, age, employment status, housing status, digital footprint indicators, velocity metrics, and a temporal month feature).
- **Target Variable (y):** $\text{fraud_bool} \in \{0,1\}$, where:
 - 1 indicates a fraudulent account opening attempt.
 - 0 indicates a legitimate account opening.

3. Machine Learning Success Metrics

Because the target class is extremely rare (1.10% prevalence), traditional metrics like global accuracy or even weighted F1-score are highly misleading. Instead, we use metrics focused heavily on the minority class and operational realities.

- **Primary Metric: True Positive Rate at a fixed 5% False Positive Rate (TPR @ 5% FPR)**
 - *Why?* This is the standardized benchmark metric proposed by the authors of the BAF dataset. Realistically, financial institutions have fixed security operations (SecOps) teams. Capping the FPR at 5% means the bank accepts that it can only manually review up to 5% of all applications. Your goal is to maximize the amount of actual fraud caught (TPR/Recall) within that 5% operational "budget".
- **Secondary Metric: Area Under the Precision-Recall Curve (PR-AUC)**
 - *Why?* Standard ROC-AUC can be deceptively optimistic because the overwhelming number of true negatives (legitimate applications) inflates the curve. PR-AUC focuses entirely on the precision-recall trade-off of the rare fraud class.
- **Fairness Metric: Equalized Odds or Demographic Parity Difference**
 - *Why?* The BAF dataset contains protected attributes like age. To avoid regulatory and ethical issues, the model's false positive rates and selection rates must remain uniform across different age brackets.



As you begin modeling, the curve above represents your biggest technical challenge. Under severe class imbalance, attempting to capture more fraud (moving to the right to increase Recall) will drastically increase your false positives, dragging down Precision and placing a massive burden on the bank's manual review team. Your job is to engineer features and select algorithms that push this frontier as far to the top-right corner as possible.

4. Business Key Performance Indicators (KPIs)

- **Fraud Capture Rate (FCR):** The absolute percentage of total fraud-related financial losses prevented by the model.
- **Customer Friction Rate:** The percentage of legitimate customers flagged by the model (this directly maps to the FPR and should ideally stay under 5%).
- **Net Cost Savings (NCS):** The ultimate dollar-value savings metric:

$$NCS = (TP \times L_{\text{fraud}}) - (FP \times C_{\text{friction}}) - ((TP + FP) \times C_{\text{review}})$$

Where:

- L_{fraud} is the average financial loss prevented per blocked fraudulent account.
- C_{friction} is the cost of losing a frustrated, legitimate customer due to a false alarm.
- C_{review} is the operational cost of an analyst manually auditing a flagged application.

This step is where your machine learning expertise needs to shine. The Bank Account Fraud (BAF) dataset is incredibly realistic because its creators left "traps" that would break naive models—specifically, **hidden missing values** and **highly skewed distributions**.

Step 2: Data Collection & Understanding

1. Dataset Overview

The **Bank Account Fraud (BAF) Base Dataset** is a realistic, large-scale, tabular dataset containing **1,000,000 rows (applications)** and **32 features** spanning an 8-month period.

Statistical & Structural Summary

- **Observations (Rows):** 1,000,000
- **Features (Columns):** 31 predictors + 1 binary target (fraud_bool)
- **Data Types:** 27 numerical features (integers and floats) and 5 categorical features.
- **Class Imbalance:** Extreme. Only **11,037 applications (1.10%)** are flagged as fraud (fraud_bool = 1), while 988,963 (98.90%) are legitimate.

- **Temporal Component:** Governed by the month column (integer values from 0 to 7). The fraud prevalence remains relatively stable month-to-month, but the feature distributions shift over time (covariate shift).

2. The "Hidden" Missing Values Trap

If you run a standard pandas check like `df.isnull().sum()`, it will return **zero missing values**. Do not be fooled! The authors encoded missing values using negative integers (specifically **-1**) for several numerical features.

Before doing any feature engineering or modeling, you must handle these -1 values (either by treating them as a distinct category, imputing them, or letting tree-based models natively handle them if configured correctly).

The primary features containing these hidden nulls are:

- `prev_address_months_count` (~71.2% missing)
- `current_address_months_count` (~0.4% missing)
- `intended_balcon_amount` (~74.3% missing)
- `bank_months_count` (~25.4% missing)
- `session_length_in_minutes` (~2.0% missing)

Variable Name	Data Type	Value Range / Allowed Values	Units / Format	Description & Critical Notes
<code>fraud_bool</code>	Binary	0, 1	Indicator	Target Variable. 1 = Fraudulent application, 0 = Legitimate.
<code>income</code>	Continuous	0.1 to 0.9	Ratio	Annual income of the applicant, normalized into deciles.
<code>customer_age</code>	Discrete	10, 20, ..., 90	Years (Rounded)	Age of the applicant, rounded to the nearest decade (acting as a proxy for privacy).

Variable Name	Data Type	Value Range / Allowed Values	Units / Format	Description & Critical Notes
employment_status	Categorical	CA, CB, CC, CD, CE, CF, CG	Encoded Category	Employment status category of the applicant.
housing_status	Categorical	BA, BB, BC, BD, BE, BF, BG	Encoded Category	Housing status (e.g., owner, renter) of the applicant.
intended_balance_amount	Continuous	-1.0 to 113.0	Currency	Initial intended balance transfer. 74% are -1 (Missing).
previous_address_months_count	Integer	-1 to 380	Months	Duration lived at the previous address. 71% are -1 (Missing).
current_address_months_count	Integer	-1 to 420	Months	Duration lived at the current address.
bank_months_count	Integer	-1 to 32	Months	Age of the applicant's existing account with the bank. 25% are -1 (Missing).
velocity_6h	Continuous	0.0 to 16000.0	Count/Rate	Number of applications from the same IP address in the last 6 hours. Highly right-skewed.
velocity_24h	Continuous	0.0 to 9000.0	Count/Rate	Number of applications from the same IP address in the last 24 hours.

Variable Name	Data Type	Value Range / Allowed Values	Units / Format	Description & Critical Notes
phone_home_valid	Binary	0, 1	Indicator	Validation status of the provided home phone number.
phone_mobile_valid	Binary	0, 1	Indicator	Validation status of the provided mobile phone number.
session_length_in_minutes	Continuous	-1.0 to 100.0	Minutes	Duration of the application session online.
device_os	Categorical	windows, macOS, linux, other, x11	OS Name	Operating system of the device used to apply.
keep_alive_session	Binary	0, 1	Indicator	Did the user session stay active (keep-alive pings sent)?
month	Discrete	0 to 7	Months	Temporal Split Variable. The month of the application.

4. Critical Distribution & Outlier Insights

- **Extreme Skewness in Velocities:** The features tracking applicant velocities (like velocity_6h and velocity_24h) have severe right-skewed distributions. A tiny fraction of IPs generate thousands of requests, which is a classic signature of botnets or coordinated fraud attacks. Traditional linear models will struggle with these outliers without a logarithmic transform or robust scaling.
- **The Age Demographic (Bias Risk):** Because customer_age is rounded to the nearest decade, it acts as a discrete ordinal variable.